

GPUMODE Lecture Study Notes: Lecture 93: Cornserve Easy, Fast and Scalable Multimodal AI

Core Problem the Lecture Addresses

This lecture explains **Cornserve**, an online serving system for efficient deployment of **any-to-any multimodal generative AI models**. The central problem is that modern generative models no longer behave like a single homogeneous text-only large language model. Instead, they contain multiple components, each with different computational behavior, resource requirements, batching opportunities, and invocation patterns.

Text-only large language models take text as input and produce text as output. Vision-language models add image encoders. More recent omni models add audio encoders, video encoders, image encoders, language-model backbones, and output generators such as audio generators or image generators. A request may use only text, text plus image, text plus audio, text plus video, or some combination of these modalities. The output may also be text, audio, image, or another modality.

The technical problem is therefore not merely “how do we make one kernel faster?” but rather:

How do we serve heterogeneous multimodal model components efficiently under heterogeneous

The lecture argues that existing serving systems are mostly **point solutions**. They work well for a subset of models and workloads, but not universally. For example, a monolithic deployment may be best for one vision-language workload, encoder-disaggregated serving may be best for another, and prefill-decode disaggregation may help latency in yet another. There is no single static deployment strategy that dominates.

Cornserve addresses this by using:

- **component-level disaggregation**, where encoders, language-model components, and generators may be deployed separately;

- **task abstractions**, where each model component is represented as a unit task or composite task;
- **component-wise profiling**, where each component is profiled independently under candidate serving paths;
- **automated planning**, where the system chooses resource allocation, colocation, disaggregation, and routing decisions;
- **a distributed runtime**, where requests are decomposed into component invocations and dispatched to the correct task executors.

The key thesis is:

Any-to-any model serving should not be hard-coded around one model architecture. The serving system should understand the model graph, request distribution, and component capacities, then automatically decide how to deploy and route requests.

Required Background

Autoregressive Language Model Inference

A text-only large language model receives an input prompt and generates output tokens autoregressively. If the prompt is tokenized into a sequence

$$x_1, x_2, \dots, x_n,$$

then the model predicts the next token by modeling

$$p(x_{n+1} \mid x_1, x_2, \dots, x_n).$$

After the next token is generated, it is appended to the context, and the model repeats:

$$p(x_{n+2} \mid x_1, x_2, \dots, x_n, x_{n+1}).$$

Thus generation is sequential at the token level. The model cannot generate token x_{t+1} until it has generated token x_t .

The inference process is usually divided into two phases:

- **Prefill**: process the entire input prompt and generate the first output token.
- **Decode**: generate subsequent tokens one at a time while reusing the key-value cache.

Prefill and Decode

Let L_{in} be the number of input tokens and L_{out} be the number of generated tokens. In transformer inference, prefill processes all input tokens in parallel across sequence positions, while decode processes one new token per request at a time.

The prefill phase performs attention over the input sequence. For a transformer layer with query, key, and value matrices,

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

attention computes

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

During prefill, Q , K , and V are computed for many prompt tokens. This tends to have relatively high arithmetic intensity and can be compute-bound.

During decode, the model generates one token per sequence. The new query attends over all previously cached keys and values. The key-value cache grows as output length grows. Decode tends to be memory-bandwidth-bound because it repeatedly reads cached K and V tensors.

A simplified distinction is:

Prefill: high parallelism, often compute-bound

Decode: low per-request parallelism, KV-cache heavy, often memory-bound

KV Cache

The **KV cache** stores the key and value tensors computed for previous tokens. For a model with N_ℓ transformer layers, H attention heads, head dimension d_h , sequence length S , and bytes per element b , the approximate KV-cache memory per request is

$$M_{\text{KV}} = 2 \cdot N_\ell \cdot H \cdot S \cdot d_h \cdot b.$$

The factor 2 appears because both keys and values are stored.

As S grows, decode becomes increasingly expensive in memory traffic. The serving system must manage this cache carefully across many concurrent requests.

Continuous Batching

In online inference, requests arrive over time. Continuous batching allows the serving engine to dynamically batch active decode steps across different requests. Instead of waiting for a fixed batch to finish, the engine can insert new requests as old requests complete.

If the active set of decoding requests at time t is $\mathcal{B}(t)$, then each decode step performs work for

$$|\mathcal{B}(t)|$$

active requests. Larger decode batches improve GPU utilization because matrix-vector-like decode work can be transformed into more efficient batched operations.

State-of-the-art language-model serving systems such as vLLM exploit continuous batching and paged KV-cache management to improve throughput.

Multimodal Input Models

A vision-language model extends a language model by adding an encoder for non-text modalities. For an image-text request, the text is tokenized normally, while the image passes through an image encoder that produces image tokens.

Let

$$T_{\text{text}} = \text{Tokenizer}(x_{\text{text}})$$

and

$$T_{\text{image}} = \text{ImageEncoder}(x_{\text{image}}).$$

The language model receives the merged sequence

$$T_{\text{LM}} = \text{Merge}(T_{\text{text}}, T_{\text{image}}).$$

The LM then performs the same autoregressive next-token prediction as in text-only serving.

For audio and video, the pattern is similar:

$$T_{\text{audio}} = \text{AudioEncoder}(x_{\text{audio}}),$$

$$T_{\text{video}} = \text{VideoEncoder}(x_{\text{video}}).$$

The multimodal encoder converts raw modality-specific data into tokens or embeddings compatible with the language model.

Any-to-Any Models

Any-to-any models generalize multimodal input models. They can accept multiple input modalities and produce multiple output modalities. For example, a model may accept text, image, audio, and video as input, then produce text and audio as output.

The lecture uses Qwen-style omni models as representative examples. These models may contain:

- text tokenizer;
- image or vision encoder;
- audio encoder;
- video encoder;
- language-model backbone;
- audio generator;
- image generator;
- other modality-specific decoders or generators.

A generic any-to-any model can be written as a graph:

$$G_{\text{model}} = (V, E),$$

where V is a set of components and E is a set of data dependencies between components.

Each request activates only a subgraph:

$$G_{\text{request}} \subseteq G_{\text{model}}.$$

For example, an image-plus-text request asking for text output may activate:

$$\text{Image Encoder} \rightarrow \text{LM}.$$

A video-plus-text request asking for audio output may activate:

$$\text{Video Encoder} \rightarrow \text{LM} \rightarrow \text{Audio Generator}.$$

A video-plus-audio-plus-text request asking for text output may activate:

$$\{\text{Video Encoder, Audio Encoder}\} \rightarrow \text{LM}.$$

This request-level heterogeneity is one of the core reasons monolithic serving becomes inefficient.

Main Concepts

Evolution from Text LLMs to Omni Models

The lecture begins by tracing the evolution of modern generative AI systems:

1. **Text-only LLMs:** systems such as early ChatGPT-style models predict the next token from a text prompt.
2. **Open model momentum:** open and open-weight models such as Llama and Mixtral motivated broader deployment of LLM inference systems.
3. **Vision-language models:** models such as GPT-4V-style systems accept image and text input.
4. **Multimodal input models:** models accept text, image, audio, or video as input.
5. **Any-to-any models:** models accept arbitrary combinations of input modalities and produce arbitrary combinations of output modalities.

This evolution increases the complexity of serving. A text-only LLM has one dominant component: the language model. A vision-language model has at least an encoder plus an LM. An omni model may have several encoders, a main autoregressive model, and output generators.

Request Heterogeneity

Requests differ in which modalities they contain and which output modalities they ask for. Let $\mathcal{M}_{\text{in}}$ be the set of possible input modalities:

$$\mathcal{M}_{\text{in}} = \{\text{text, image, audio, video}\}.$$

Let $\mathcal{M}_{\text{out}}$ be the set of possible output modalities:

$$\mathcal{M}_{\text{out}} = \{\text{text, audio, image}\}.$$

A request r can be described by

$$r = (S_{\text{in}}(r), S_{\text{out}}(r)),$$

where

$$S_{\text{in}}(r) \subseteq \mathcal{M}_{\text{in}}, \quad S_{\text{out}}(r) \subseteq \mathcal{M}_{\text{out}}.$$

The invoked components are a function of this request type:

$$C(r) = f(S_{\text{in}}(r), S_{\text{out}}(r)).$$

For example:

$$C(\text{text only} \rightarrow \text{text}) = \{\text{LM}\},$$

$$C(\text{image plus text} \rightarrow \text{text}) = \{\text{Image Encoder, LM}\},$$

$$C(\text{video plus text} \rightarrow \text{audio}) = \{\text{Video Encoder, LM, Audio Generator}\}.$$

This makes fixed monolithic execution wasteful. If the full model is loaded as one service, the system may reserve resources for components that are unused by many requests.

Component Heterogeneity

Components differ in computational intensity, memory footprint, batching behavior, latency profile, and throughput. The lecture emphasizes that in any-to-any models, components can have drastically different serving capacities.

Let component i have service capacity μ_i , measured in requests per second for a specific request path and batch regime. If a monolithic deployment forces all requests through a single sequential execution pipeline, then the effective throughput is limited by the slowest heavily used component:

$$\mu_{\text{monolithic}} \approx \min_i \mu_i.$$

This is especially problematic when one output generator, such as an audio generator, is much slower than encoders or the LM. In the lecture's example, the audio generator can become the bottleneck of the entire monolithic server.

Component heterogeneity motivates disaggregation:

$$\text{Model} \rightarrow \{\text{Encoder Workers, LM Workers, Generator Workers}\}.$$

Each component can then be independently replicated, batched, profiled, and scheduled.

Monolithic Serving

In monolithic serving, all components live in one server process or one tightly coupled deployment. For a vision-language model, a monolithic server might contain:

Image Encoder + LM.

For an omni model, a monolithic server might contain:

Vision Encoder + Audio Encoder + Video Encoder + LM + Audio Generator.

The advantage is simplicity. The disadvantages are:

- components cannot be independently scaled;
- unused components still occupy memory;
- only one component may make progress at a time in naive implementations;
- component-specific batching is limited;
- slow components bottleneck the whole server;
- large omni models may trigger out-of-memory failures.

Monolithic deployment is not always bad. The lecture explicitly notes that for some multimodal input workloads, monolithic serving is surprisingly robust and may be the best option. Cornserve's goal is not to always disaggregate. Its goal is to decide when disaggregation helps and when it does not.

Prefill-Decode Disaggregation

For text-only LLMs, an important deployment strategy is **prefill-decode disaggregation**, abbreviated PD disaggregation.

In a standard continuous batching engine, a request can be interrupted by the prefill phase of a later request. Suppose request 1 is decoding. When request 2 arrives, the engine performs prefill for request 2, which can interrupt request 1's decode progress. Request 3 may later interrupt both.

PD disaggregation avoids this by dedicating some workers to prefill and others to decode:

Prefill Worker → KV Cache Transfer → Decode Worker.

The prefill worker computes the initial KV cache, then transfers it to the decode worker. The decode worker continuously batches decode tokens and is less frequently interrupted by large prefill work.

The approximate pipeline is:

$$r \rightarrow \text{Prefill}(r) \rightarrow \text{TransferKV}(r) \rightarrow \text{Decode}(r).$$

This can improve latency isolation and decode efficiency, though it introduces KV-transfer overhead.

Encoder-Prefill-Decode Disaggregation

For multimodal input models, there are three natural phases:

$$\text{Encode} \rightarrow \text{Prefill} \rightarrow \text{Decode}.$$

Different systems have explored different slicing choices:

- **Monolithic**: encoder, prefill, and decode are colocated.
- **EP/D**: encoder and prefill are colocated, decode is disaggregated.
- **E/PD**: encoder is disaggregated, prefill and decode are colocated.
- **E/P/D**: encoder, prefill, and decode are all disaggregated.

The lecture presents evaluation results showing that no single strategy dominates. Depending on image resolution, image count, input length, output length, model size, and GPU count, different deployment strategies win.

This motivates automated planning rather than manual system selection.

Cornserve's Main Design Idea

Cornserve treats the model as a graph of deployable components. It profiles component capacities, considers possible execution paths, and solves a planning problem to decide:

- how many replicas of each component to create;
- which components should be colocated;
- which components should be disaggregated;
- how requests should be routed;
- how many GPUs are needed to satisfy the target workload;
- how to maximize throughput or serving capacity under resource constraints.

A simplified representation is:

Model Definition+Workflow Distribution $\rightarrow$ Planner $\rightarrow$ Deployment Plan $\rightarrow$ Distributed Runtime.

Cornserve System Architecture

Gateway

The **gateway** is the entry point for model deployment requests and inference requests.

For model deployment, it receives:

(model definition, intended serving workflow) .

For inference, it receives user requests containing modalities and output requirements. It then determines which component invocations are needed.

The gateway is responsible for initiating symbolic or recorded execution of the model workflow so that the runtime knows which tasks are required for the current request.

Planner

The planner is responsible for resource allocation and deployment strategy. It receives the model graph and the target workload distribution.

The planner performs:

1. **graph extraction**: derive possible executable component graphs from the task abstraction;
2. **path analysis**: enumerate possible execution paths for each request type;
3. **profiling**: estimate component capacities under candidate paths;
4. **optimization**: solve for the best deployment plan;
5. **resource assignment**: decide replicas, placement, disaggregation, and routing.

The planner is the part of Cornserve that distinguishes it from systems that only implement one fixed serving strategy.

Task Managers

After the planner creates a deployment plan, Cornserve launches **task managers**. A task manager controls one task type, such as an image encoder, LM executor, or audio generator.

A task manager is responsible for:

- spawning task executors;
- managing replicas;

- receiving dispatched invocations;
- coordinating execution within its component type;
- providing a scalable interface to the runtime.

Task Executors

A **task executor** runs the actual computation. It may be a custom encoder executor, a custom generator executor, or an LM executor backed by an existing serving engine such as vLLM.

Examples include:

Image Encoder Executor, Video Encoder Executor, LM Executor, Audio Generator Executor.

The lecture emphasizes that Cornserve is modular. It does not need to rewrite all language-model serving internals. Instead, it can reuse existing serving engines for the LM component.

Task Dispatcher

The **task dispatcher** receives recorded component invocations from the gateway and dispatches them to the correct task managers and task executors.

For a request with video and text input asking for text output, the dispatcher may send work to:

Video Encoder Task Manager and LM Task Manager.

For a request with video and text input asking for audio output, it may send work to:

Video Encoder Task Manager → LM Task Manager → Audio Generator Task Manager.

Distributed Runtime

The distributed runtime is the execution layer. It runs on Kubernetes and uses task managers and task executors to execute component invocations. The lecture states that the implementation contains approximately 15,000 lines of Python, with custom encoder and generator executors, an LM executor based on vLLM, Kubernetes deployment support, and OpenTelemetry observability.

The runtime design goal is to make component executors behave like scalable, stateless microservices where possible.

Task Abstraction

Unit Tasks

A **unit task** is the smallest deployable and scalable unit in Cornserve. It executes in a single Kubernetes pod and represents a basic model component.

Examples:

Unit Task $\in$ {Image Encoder, Audio Encoder, Video Encoder, LM, Audio Generator}.

A unit task has several important properties:

- it is the smallest scaling unit;
- it can be independently replicated;
- it can be independently profiled;
- it can be independently placed on GPU resources;
- it exposes a component-level invocation interface.

Composite Tasks

A **composite task** is a Python-defined composition of one or more unit tasks. It expresses a higher-level model workflow.

For example, a multimodal language-model composite task may include:

MLM Composite = {Modality Encoder, LM}.

An omni composite task may include:

Omni Composite = {Vision Encoder, Audio Encoder, Video Encoder, LM, Audio Generator}.

Composite tasks allow users to define model-specific structure without hard-coding the system for one architecture.

Example: Multimodal Language Model Composite Task

A simplified Python-style representation is:

```
class MultimodalLMTask:
    def __init__(
        self,
        model_id: str,
```

```

        modalities: list[str],
        encoder_disaggregated: bool,
        encoder_model_id: str | None = None,
    ):
        self.model_id = model_id
        self.modalities = modalities
        self.encoder_disaggregated = encoder_disaggregated
        self.encoder_model_id = encoder_model_id

        self.encoders = {}
        for modality in modalities:
            self.encoders[modality] = UnitTask(
                name=f"{modality}_encoder",
                model_id=encoder_model_id or model_id,
            )

        self.lm = UnitTask(
            name="language_model",
            model_id=model_id,
        )

    def forward(self, request):
        encoded_tokens = []

        if request.has_image():
            encoded_tokens.append(
                self.encoders["image"](request.image)
            )

        if request.has_audio():
            encoded_tokens.append(
                self.encoders["audio"](request.audio)
            )

        text_tokens = tokenize(request.text)

        lm_input = merge_tokens(text_tokens, encoded_tokens)
        return self.lm(lm_input)

```

This code captures several system ideas:

- each modality has a separate encoder unit task;
- the language model is another unit task;
- only encoders needed by the request are invoked;
- encoder disaggregation can be controlled by configuration;

- model variants can share encoders if they use the same encoder model.

Hardware and Runtime Meaning of the Code

The call

```
self.encoders["image"](request.image)
```

does not necessarily mean that the image encoder runs inside the same process as the LM. In Cornserve, this may become a remote invocation to an image encoder executor. That executor may live on a different GPU from the LM executor.

The call

```
self.lm(lm_input)
```

may be routed to a language-model serving engine such as vLLM, which performs continuous batching and KV-cache management.

The branching logic around request modalities is important because different requests activate different subgraphs. Cornserve records these invocations and dispatches them to the data plane.

Encoder Sharing Across Model Variants

The lecture gives an example involving model variants such as a smaller and larger Gemma-style multimodal model. If both variants use the same vision encoder, Cornserve can share the encoder component across deployments.

Suppose two models are:

$$M_1 = E_{\text{vision}} + L_1,$$

$$M_2 = E_{\text{vision}} + L_2.$$

Instead of deploying two copies of E_{vision} , Cornserve may deploy:

$$E_{\text{vision}} + \{L_1, L_2\}.$$

This saves GPU memory and can improve utilization if the shared encoder is not the bottleneck.

Record and Replay Invocation Mechanism

Why Invocation Recording is Needed

In an any-to-any model, the system cannot assume that every component is needed for every request. A request may include video but no image. Another may include image and audio. Some outputs require an audio generator, while others require only the LM.

Cornserve needs to determine the exact component invocation set:

$$I(r) = \{i_1, i_2, \dots, i_k\},$$

where each i_j is a unit-task invocation needed by request r .

A naive approach would execute Python code directly, but direct execution would run components immediately rather than produce a dispatchable graph. Cornserve instead uses a **record and replay** mechanism.

Recording

During recording, Cornserve symbolically executes or traces the composite task logic. When the code reaches a unit-task call, the system records the invocation instead of executing it immediately.

Conceptually:

```
with record_invocations() as tape:
    output = composite_task.forward(request)
```

```
invocations = tape.invocations
```

If the request has video and text but no image, the recorded tape may contain:

```
[
    Invocation(task="video_encoder", input=request.video),
    Invocation(task="language_model", input=video_tokens_and_text_tokens),
]
```

It does not contain an image-encoder invocation because the image branch was not taken.

Replay

During replay, the task dispatcher takes the recorded invocations and dispatches them to the distributed data plane:

```
for invocation in invocations:
    dispatcher.dispatch(invocation)
```

The important distinction is:

Record = discover the required component calls,

Replay = execute those calls on distributed executors.

Parallelism Benefit

The lecture discussion clarifies that independent components can be spawned independently. For example, if a request contains both image and audio, then the image encoder and audio encoder may be independent:

Image Encoder || Audio Encoder.

A sequential Python branch structure may obscure this parallelism. Recording the invocations allows the runtime to see the component-level graph and schedule independent tasks concurrently.

For a request with image, audio, and text, the dependency graph may be:

$\{\text{Image Encoder, Audio Encoder}\} \rightarrow \text{Merge} \rightarrow \text{LM}.$

The encoders can execute in parallel because neither depends on the other.

Visual Concept

Draw a request entering the gateway. From the gateway, draw a “recording tape” that captures only the invoked tasks. Then draw arrows from the tape into separate task managers: image encoder, audio encoder, video encoder, LM, and audio generator. Show unused components grayed out.

Planner

Planner Inputs

The planner accepts two main inputs:

1. **Model computation graph:** defined through Cornserve’s task abstraction.
2. **Target workflow:** a distribution over request types and request shapes.

Let the workflow contain request classes $r_1, r_2, \dots, r_m$. Each request class has arrival rate λ_j :

λ_j = arrival rate of request type r_j .

The total arrival rate is:

$$\lambda_{\text{total}} = \sum_{j=1}^m \lambda_j.$$

The fraction of request type j is:

$$p_j = \frac{\lambda_j}{\lambda_{\text{total}}}.$$

The planner must decide how to serve this distribution efficiently.

Executor Graph

From the model computation graph, the planner creates an **executor graph**. The executor graph contains possible deployment choices, such as:

- disaggregated encoder executors;
- disaggregated LM executors;
- monolithic executors containing encoder plus LM;
- generator executors;
- colocated component groups.

For a toy vision-language model, possible executors might be:

$$E_{\text{img}}, \quad L_{\text{LM}}, \quad M_{\text{mono}} = E_{\text{img}} + L_{\text{LM}}.$$

Path Analysis

For each request type, the planner enumerates possible execution paths.

For a text-only request, possible paths may be:

$$P_{\text{text},1} = L_{\text{LM}},$$

$$P_{\text{text},2} = M_{\text{mono}}.$$

For an image-plus-text request, possible paths may be:

$$P_{\text{img},1} = M_{\text{mono}},$$

$$P_{\text{img},2} = E_{\text{img}} \rightarrow L_{\text{LM}},$$

$$P_{\text{img},3} = E_{\text{img}} \rightarrow M_{\text{mono}}.$$

The third path is possible if the monolithic deployment can accept already encoded image tokens or if it contains a compatible LM execution route. The broader point is that a request can sometimes be routed through multiple valid combinations of executors.

Profiling

The planner profiles the serving capacity of each component and path. For executor e and request class j , define:

$$\mu_{e,j} = \text{capacity of executor } e \text{ for request class } j.$$

Capacity depends on request dimensions such as:

- image resolution;
- number of images;
- video frame count;
- audio length;
- input token length;
- output token length;
- batch size;
- model size;
- tensor parallelism degree;
- GPU type.

The planner cannot rely only on static intuition. It must profile because the best deployment strategy changes across workloads.

Optimization Objective

One objective mentioned in the lecture is minimizing the number of GPUs needed to serve a workload. A simplified optimization objective is:

$$\min \sum_{e \in \mathcal{E}} g_e x_e,$$

where:

- $\mathcal{E}$ is the set of executor types;
- x_e is the number of replicas of executor e ;
- g_e is the number of GPUs required by one replica of executor e .

The system must satisfy demand constraints. If request type j can be routed over paths $\mathcal{P}_j$, and $f_{j,p}$ is the flow of request type j routed over path p , then:

$$\sum_{p \in \mathcal{P}_j} f_{j,p} = \lambda_j.$$

Executor capacity constraints ensure no executor is overloaded:

$$\sum_j \sum_{p \in \mathcal{P}_j: e \in p} a_{e,j,p} f_{j,p} \leq x_e \mu_e.$$

Here $a_{e,j,p}$ represents the amount of executor e 's service capacity consumed by request type j on path p .

Multicommodity Network Design View

The lecture states that the planning problem is formulated as a **multicommodity network design problem**. Each request type behaves like a commodity that must be routed through a network of possible executors.

In this analogy:

- commodities are request classes;
- nodes are task executors or component states;
- edges are possible transitions between components;
- capacities are executor throughputs;
- costs are GPU resource usage or latency penalties;
- flows are fractions of traffic routed along paths.

A generic network-design formulation is:

$$\min_{x,f} \sum_{e \in \mathcal{E}} c_e x_e + \sum_j \sum_{p \in \mathcal{P}_j} d_{j,p} f_{j,p},$$

subject to:

$$\sum_{p \in \mathcal{P}_j} f_{j,p} = \lambda_j, \quad \forall j,$$

$$\sum_j \sum_{p \in \mathcal{P}_j: e \in p} f_{j,p} \leq \mu_e x_e, \quad \forall e,$$

$$x_e \in \mathbb{Z}_{\geq 0}, \quad f_{j,p} \geq 0.$$

Here:

- c_e is the resource cost of executor e ;
- $d_{j,p}$ is an optional routing cost for request type j on path p ;
- x_e is the integer number of deployed replicas;
- $f_{j,p}$ is the request flow assigned to path p .

The exact Cornserve formulation may include additional constraints for colocation, tensor parallelism, memory capacity, batching, and data-transfer overhead.

Hardware-Level Explanation

GPU Execution Model

A GPU consists of many streaming multiprocessors, abbreviated SMs. A kernel launch creates a grid of thread blocks. Each thread block is assigned to an SM. Threads inside a block are grouped into warps, usually 32 threads per warp on NVIDIA GPUs.

The hierarchy is:

$$\text{Grid} \rightarrow \text{Thread Blocks} \rightarrow \text{Warps} \rightarrow \text{Threads}.$$

Each SM contains:

- warp schedulers;
- CUDA cores;
- tensor cores;

- registers;
- shared memory;
- L1 cache;
- load-store units;
- special-function units.

Global memory is high-capacity but high-latency. L2 cache is shared across SMs. Shared memory is explicitly managed within an SM and is much faster than global memory. Registers are private to threads and are the fastest storage.

Why LM Decode is Often Memory-Bound

During decode, each request contributes one new query token. The attention computation must read previous keys and values from the KV cache.

For one layer, the approximate attention score computation for one query is:

$$qK^\top,$$

where $q \in \mathbb{R}^{d_h}$ and $K \in \mathbb{R}^{S \times d_h}$. This requires reading Sd_h key elements and then reading Sd_h value elements for the weighted sum.

Approximate bytes read per layer per request are:

$$B_{\text{decode,layer}} \approx 2Sd_h bH,$$

where:

- S is current sequence length;
- d_h is head dimension;
- b is bytes per element;
- H is number of heads;
- factor 2 accounts for keys and values.

As S grows, memory traffic grows linearly. Since each request has only one new token, compute parallelism per request is limited. Continuous batching is therefore critical to expose enough parallel work to the GPU.

Why Prefill is Often Compute-Bound

During prefill, the model processes many prompt tokens at once. Matrix multiplications such as

$$XW_Q, \quad XW_K, \quad XW_V, \quad XW_O$$

are large GEMMs. Large GEMMs have high arithmetic intensity because each loaded weight tile can be reused across many tokens and batch elements.

Arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

If arithmetic intensity is high enough, performance approaches the GPU’s compute roof. If it is low, performance is limited by memory bandwidth.

Roofline Model

The roofline model estimates attainable performance as:

$$P_{\text{attainable}} = \min(P_{\text{peak}}, I \cdot B_{\text{mem}}),$$

where:

- P_{peak} is peak compute throughput;
- I is arithmetic intensity in FLOPs per byte;
- B_{mem} is memory bandwidth.

For prefill, I tends to be high, so the kernel can be compute-bound:

$$P_{\text{prefill}} \approx P_{\text{peak}}.$$

For decode, I tends to be lower due to KV-cache reads:

$$P_{\text{decode}} \approx I_{\text{decode}} B_{\text{mem}}.$$

This difference is the reason PD disaggregation can be useful: prefill and decode stress the GPU differently.

Encoder Hardware Behavior

Multimodal encoders are often transformer-like or convolutional networks. Vision encoders may consist of patch embedding, attention blocks, MLPs, and projection layers. Audio encoders may use convolutional front ends, attention, or other sequence models. Video encoders process frames and temporal features.

The lecture notes that multimodal encoders are often smaller than the LM and can be compute-bound attention workloads. However, their runtime cost depends strongly on modality shape.

For an image with resolution $H_{\text{img}} \times W_{\text{img}}$ and patch size P , the number of image patches is approximately:

$$N_{\text{patch}} = \frac{H_{\text{img}}}{P} \cdot \frac{W_{\text{img}}}{P}.$$

If the encoder uses full attention over patches, attention complexity is:

$$O(N_{\text{patch}}^2 d).$$

For video with F frames, the number of visual tokens may scale roughly as:

$$N_{\text{video}} \approx F \cdot \frac{H_{\text{frame}}}{P} \cdot \frac{W_{\text{frame}}}{P}.$$

Thus video can become expensive very quickly, especially if temporal and spatial tokens are jointly attended.

Generator Hardware Behavior

Any-to-any models introduce output generators. The lecture highlights audio generation as a major bottleneck in some omni models. Earlier audio generators may use diffusion-style generation, while newer designs may use convolutional architectures that are faster.

A diffusion-based generator may require many denoising steps:

$$z_{t-1} = D_{\theta}(z_t, c, t), \quad t = T, T-1, \dots, 1,$$

where:

- z_t is the noisy latent at step t ;
- c is conditioning from the LM or multimodal context;
- D_{θ} is the denoising network;
- T is the number of denoising steps.

Total cost scales with T :

$$C_{\text{diffusion}} \approx T \cdot C_{D_{\theta}}.$$

If T is large, audio generation can dominate request latency and reduce throughput. A convolutional or autoregressive generator may have different scaling and better serving characteristics.

Tensor Parallelism and GPU Count

The lecture notes that for some experiments, the LM component was too large to fit on one GPU and required tensor parallelism degree 2. If an LM replica uses tensor parallelism degree t , then one LM replica consumes t GPUs:

$$g_{\text{LM replica}} = t.$$

If a monolithic deployment contains the LM and uses tensor parallelism degree 2, then one monolithic replica also consumes 2 GPUs. On 8 GPUs, this permits:

$$\frac{8}{2} = 4$$

monolithic replicas.

If encoder replicas use one GPU each and LM replicas use two GPUs each, then a deployment with 2 encoders and 3 LM replicas consumes:

$$2 \cdot 1 + 3 \cdot 2 = 8$$

GPUs.

This arithmetic is central to comparing deployment options fairly.

Mathematical Reasoning

Throughput of a Pipeline

For a sequential monolithic pipeline with components $1, 2, \dots, k$, if component i has service rate μ_i , then the pipeline throughput is limited by the bottleneck:

$$\mu_{\text{pipeline}} = \min_i \mu_i.$$

If the audio generator is slow, then:

$$\mu_{\text{pipeline}} \approx \mu_{\text{audio generator}}.$$

Disaggregation allows independent scaling. If component i has x_i replicas, its aggregate capacity is approximately:

$$\mu_i^{\text{total}} = x_i \mu_i.$$

For a stable system with arrival rate λ_i to component i , we need:

$$\lambda_i < x_i \mu_i.$$

Request Mixture and Component Load

Let request type j have arrival rate λ_j . Let $u_{i,j}$ indicate whether request type j uses component i :

$$u_{i,j} = \begin{cases} 1, & \text{if request type } j \text{ uses component } i, \\ 0, & \text{otherwise.} \end{cases}$$

Then the arrival rate to component i is:

$$\lambda_i = \sum_j u_{i,j} \lambda_j.$$

For example, suppose 20% of requests require audio output. If total arrival rate is λ , then the audio generator arrival rate is:

$$\lambda_{\text{audio gen}} = 0.2\lambda.$$

The LM may still receive all requests:

$$\lambda_{\text{LM}} = \lambda.$$

This means the optimal number of audio-generator replicas may be very different from the optimal number of LM replicas.

Latency Decomposition

End-to-end latency for request r can be decomposed as:

$$T(r) = T_{\text{queue}}(r) + T_{\text{encode}}(r) + T_{\text{prefill}}(r) + T_{\text{decode}}(r) + T_{\text{generate}}(r) + T_{\text{transfer}}(r).$$

Not all terms are present for all requests. For a text-only request:

$$T(r) = T_{\text{queue}} + T_{\text{prefill}} + T_{\text{decode}}.$$

For a video-to-audio request:

$$T(r) = T_{\text{queue}} + T_{\text{video encode}} + T_{\text{prefill}} + T_{\text{decode}} + T_{\text{audio generate}} + T_{\text{transfer}}.$$

Disaggregation may reduce queueing and improve batching, but it can increase transfer cost. The planner must balance:

$$\text{Batching benefit} + \text{parallelism benefit} - \text{communication overhead}.$$

Compute Cost of Attention

For a sequence length S and hidden dimension d , full self-attention has approximate compute cost:

$$C_{\text{attn}} = O(S^2 d).$$

The feed-forward network cost per transformer layer is roughly:

$$C_{\text{ffn}} = O(S d d_{\text{ff}}),$$

where d_{ff} is the intermediate dimension.

In prefill, $S = L_{\text{in}}$ or includes multimodal tokens:

$$S_{\text{prefill}} = L_{\text{text}} + L_{\text{image}} + L_{\text{audio}} + L_{\text{video}}.$$

Thus multimodal inputs can significantly increase prefill cost:

$$C_{\text{prefill}} = O(S_{\text{prefill}}^2 d) + O(S_{\text{prefill}} d d_{\text{ff}}).$$

Image Token Count

If an image of size $H \times W$ is divided into patches of size $P \times P$, then image token count is:

$$L_{\text{image}} = \left\lceil \frac{H}{P} \right\rceil \left\lceil \frac{W}{P} \right\rceil.$$

If a request contains N_{img} images, then:

$$L_{\text{image, total}} = N_{\text{img}} L_{\text{image}}.$$

This explains why image resolution and expected image count can change the best deployment strategy.

Video Token Count

For video with F sampled frames, frame resolution $H \times W$, and patch size P , a simplified token count is:

$$L_{\text{video}} = F \left\lceil \frac{H}{P} \right\rceil \left\lceil \frac{W}{P} \right\rceil.$$

If the video encoder performs attention over all spatiotemporal tokens, complexity may scale as:

$$O(L_{\text{video}}^2 d).$$

If the encoder factorizes spatial and temporal attention, cost may be reduced, but video still introduces much higher variance than text-only requests.

Batching Benefit

If a component processes requests one by one, throughput is limited. With batching, a component processes B requests together. For GEMM-heavy workloads, increasing batch size can improve hardware utilization.

Let $T(B)$ be the time to process batch size B . Throughput is:

$$\text{Throughput}(B) = \frac{B}{T(B)}.$$

If batching is efficient, then:

$$T(B) < BT(1),$$

so:

$$\text{Throughput}(B) > \text{Throughput}(1).$$

Monolithic deployments can limit batching because components are tied together. Disaggregated deployments allow each component to batch independently.

Code Walkthrough

Minimal Any-to-Any Model Workflow

The following pseudocode reconstructs the style of model workflow Cornserve needs to support:

```

class OmniWorkflow:
    def __init__(self):
        self.image_encoder = UnitTask("image_encoder")
        self.audio_encoder = UnitTask("audio_encoder")
        self.video_encoder = UnitTask("video_encoder")
        self.lm = UnitTask("language_model")
        self.audio_generator = UnitTask("audio_generator")

    def forward(self, request):
        context_tokens = []

        if request.text is not None:
            context_tokens.extend(tokenize(request.text))

        if request.image is not None:
            image_tokens = self.image_encoder(request.image)
            context_tokens.extend(image_tokens)

        if request.audio is not None:
            audio_tokens = self.audio_encoder(request.audio)
            context_tokens.extend(audio_tokens)

        if request.video is not None:
            video_tokens = self.video_encoder(request.video)
            context_tokens.extend(video_tokens)

        text_output = self.lm(context_tokens)

        if request.output_modality == "audio":
            audio_output = self.audio_generator(text_output)
            return audio_output

        return text_output

```

Line-by-Line Explanation

- `UnitTask("image_encoder")` declares a separately deployable image encoder component.
- `UnitTask("audio_encoder")` declares a separately deployable audio encoder component.
- `UnitTask("video_encoder")` declares a separately deployable video encoder component.
- `UnitTask("language_model")` represents the main autoregressive LM, likely backed by a high-performance inference engine.

- `UnitTask("audio_generator")` represents a modality output generator.
- Each `if` condition activates only the components required for the request.
- The LM receives merged multimodal tokens.
- The audio generator runs only when audio output is requested.

Hardware Behavior

The image, audio, and video encoders may run on separate GPUs or separate executor replicas. If a request contains multiple independent modalities, these encoders can run concurrently.

The LM is likely the most memory-intensive component. It stores weights, activation buffers, and KV cache. If the LM is large, it may use tensor parallelism across multiple GPUs.

The audio generator may have different performance characteristics from the LM. If it is diffusion-based, it may require many sequential denoising steps, making it latency-heavy. If it is convolution-based, it may be more parallel and faster.

Record and Replay Pseudocode

```
class InvocationRecorder:
    def __init__(self):
        self.invocations = []

    def record(self, task_name, payload):
        handle = FutureHandle(task_name)
        self.invocations.append((task_name, payload, handle))
        return handle

def unit_task_call(task_name, payload):
    recorder = current_recorder.get()

    if recorder is not None:
        return recorder.record(task_name, payload)

    return execute_locally(task_name, payload)

def serve_request(workflow, request):
    recorder = InvocationRecorder()

    with use_recorder(recorder):
        symbolic_output = workflow.forward(request)

    return dispatcher.replay(
```

```

        recorder.invocations,
        symbolic_output,
    )

```

Explanation of Record and Replay Code

- `InvocationRecorder` stores component calls.
- `unit_task_call` checks whether execution is in recording mode.
- If recording is active, it records the invocation and returns a future-like handle.
- If recording is not active, it executes normally.
- `serve_request` first records the workflow-specific component calls.
- The dispatcher then replays those invocations in the distributed runtime.

The runtime can analyze the recorded invocation graph to find independent tasks. For example:

```

image encoder || audio encoder || video encoder.

```

Only after these complete does the LM need the merged token sequence.

Simplified Planner Pseudocode

```

def plan_deployment(model_graph, workflow, gpu_budget):
    executor_graph = build_executor_graph(model_graph)

    all_paths = {}
    for request_type in workflow.request_types:
        all_paths[request_type] = enumerate_paths(
            executor_graph,
            request_type,
        )

    profiles = {}
    for executor in executor_graph.executors:
        for request_type in workflow.request_types:
            profiles[(executor, request_type)] = profile_capacity(
                executor,
                request_type,
            )

    solution = solve_network_design(
        executor_graph=executor_graph,

```

```

        workflow=workflow,
        paths=all_paths,
        profiles=profiles,
        gpu_budget=gpu_budget,
    )

    return solution

```

Planner Code Explanation

- `build_executor_graph` creates candidate deployment choices.
- `enumerate_paths` identifies possible routes for each request type.
- `profile_capacity` measures or estimates throughput for each component and request type.
- `solve_network_design` chooses replicas and traffic routing.
- The result is a deployment plan rather than merely a static model configuration.

Performance Intuition

Why Monolithic Serving Can Be Slow for Omni Models

In monolithic serving, all components are tied together. If the server processes one component at a time, then while the audio generator is running, the LM may not make progress. While the LM is running, encoders may not make progress. This underutilizes the system.

For a request requiring video input and audio output, monolithic execution resembles:

Video Encode → LM Prefill → LM Decode → Audio Generate.

If another request needs only text output, it may still queue behind expensive video or audio work.

Why Disaggregation Helps

Disaggregation permits different components to run independently and batch independently:

Encoder Pool LM Pool Generator Pool.

This improves:

- **resource specialization**: allocate more GPUs to slow components;
- **batching**: each component batches its own compatible requests;
- **parallelism**: independent encoders can run concurrently;
- **latency isolation**: slow output generation does not block all LM progress;
- **memory efficiency**: not every replica must load every component.

Why Disaggregation Does Not Always Help

Disaggregation introduces overheads:

- communication between components;
- serialization and deserialization of intermediate tensors;
- network transfer or inter-process transfer;
- additional scheduling complexity;
- possible loss of locality;
- more difficult failure handling.

For small encoders or short outputs, the overhead may exceed the benefit. This is why the lecture shows cases where monolithic serving is best.

The planner must compare:

$$\Delta_{\text{benefit}} = \text{batching gain} + \text{parallelism gain} + \text{scaling gain}$$

against

$$\Delta_{\text{cost}} = \text{transfer overhead} + \text{scheduling overhead} + \text{resource fragmentation}.$$

Disaggregation is worthwhile when:

$$\Delta_{\text{benefit}} > \Delta_{\text{cost}}.$$

Continuous Batching Within Modalities

The lecture clarifies that gains over monolithic baselines come partly from continuous batching **within** components, not necessarily across modalities. For example:

LM continuous batching

and

Audio generator continuous batching

can both be useful, but a monolithic deployment may prevent both from being fully exploited simultaneously.

If the LM and audio generator are separate services, the LM can continuously batch text decode while the audio generator continuously batches audio generation for the subset of requests requiring audio output.

Bottleneck Shift

Suppose a monolithic omni model has component service rates:

$$\mu_{\text{vision}} = 100, \quad \mu_{\text{LM}} = 40, \quad \mu_{\text{audio gen}} = 10.$$

The monolithic bottleneck is:

$$\mu_{\text{mono}} = 10.$$

If only 20% of requests need audio output, then disaggregation allows the audio generator to process only that subset:

$$\lambda_{\text{audio gen}} = 0.2\lambda.$$

This means the audio generator no longer bottlenecks all requests. It bottlenecks only audio-output requests.

Why Workload Shape Matters

The best deployment depends on:

- image resolution;
- expected image count;
- input text length;
- output text length;
- percentage of audio-output requests;
- video frequency;
- audio length;
- component model sizes;

- GPU memory limits;
- tensor parallelism requirements.

For example, if image inputs are rare, disaggregating image encoders may not help. If image inputs are frequent and high resolution, dedicated image encoders may improve throughput. If output lengths are long, decode dominates. If audio generation is slow and frequent, generator scaling becomes critical.

Evaluation Notes

Evaluation Setup

The lecture describes evaluations on AWS GPU nodes with A100 GPUs. Each node contains 8 GPUs interconnected by NVLink. The evaluation metrics include throughput and latency.

The evaluated model and workload categories include:

- multimodal output models such as Qwen2.5-Omni, Qwen3-Omni, and Qwen-Image-style models;
- multimodal input models such as InternVL and Qwen2.5-VL;
- real-world multimodal request traces containing text, image, audio, and video;
- controlled synthetic benchmarks varying image resolution, image count, input length, and output length;
- distributed-runtime overhead measurements.

Qwen2.5-Omni Serving

For Qwen2.5-Omni-style serving, Cornserve is compared against a monolithic Hugging Face Transformers-based baseline. The workload uses a real-world multimodal dataset containing text, images, videos, and audio inputs. The lecture states that 20% of requests are configured to require audio output.

The reported throughput improvements are:

$$3.09 \times \quad \text{on 8 A100 GPUs,}$$

and

$$3.81 \times \quad \text{on 16 A100 GPUs.}$$

The system also improves latency, as shown by a latency CDF in the lecture. The reason is that monolithic serving prevents independent progress and independent continuous batching across major components.

Out-of-Memory Avoidance

The lecture notes that Cornserve can run larger omni models such as Qwen3-Omni-style deployments that may fail in monolithic deployment due to GPU out-of-memory errors.

The memory reason is straightforward. If a monolithic server must load all components on the same GPU group, memory usage is approximately:

$$M_{\text{mono}} = M_{\text{encoders}} + M_{\text{LM}} + M_{\text{generators}} + M_{\text{KV}} + M_{\text{workspace}}.$$

Disaggregation permits spreading this across component-specific workers:

$$M_{\text{encoder worker}} = M_{\text{encoders}} + M_{\text{encoder workspace}},$$

$$M_{\text{LM worker}} = M_{\text{LM}} + M_{\text{KV}} + M_{\text{LM workspace}},$$

$$M_{\text{generator worker}} = M_{\text{generators}} + M_{\text{generator workspace}}.$$

This can make models deployable even when the full monolithic model does not fit.

InternVL Deployment Sweep

For multimodal input models, the lecture shows that deployment strategies vary across workloads. Some workloads favor encoder disaggregation. Others favor monolithic serving. Some are close across several strategies.

For controlled InternVL-style workloads, the planner can:

- match the best existing strategy;
- choose encoder-disaggregated deployment when appropriate;
- choose monolithic deployment when appropriate;
- surpass fixed strategies by mixing routes, such as routing a fraction of requests through dedicated encoders and others through monolithic replicas.

A particularly interesting result is that the planner may produce a hybrid deployment:

4 encoder replicas + 6 monolithic deployments,

with a certain percentage of requests routed to dedicated encoders. This can outperform pure monolithic or pure disaggregated baselines.

Key Evaluation Lesson

The main evaluation lesson is:

The best serving strategy depends on the model and workload. A fixed serving architecture is not enough for the diversity of any-to-any multimodal models.

Comparison to Existing Systems

vLLM

vLLM is a state-of-the-art language-model inference engine known for continuous batching and efficient KV-cache management. Cornserve can use vLLM as its LM executor rather than reimplementing all LM serving features.

This modularity matters because LM serving engines evolve quickly. Cornserve's contribution is not to replace every kernel or cache-management algorithm. It provides a higher-level system for component disaggregation, planning, and orchestration.

SGLang and TensorRT-LLM

The lecture discussion notes that Cornserve's executor model could integrate with other inference engines such as SGLang or TensorRT-LLM. The point is to reuse existing optimized implementations wherever possible.

A Cornserve LM executor could be:

LM Executor = vLLM backend

or

LM Executor = SGLang backend

or

LM Executor = TensorRT-LLM backend.

Omni-Specific Serving Systems

The lecture mentions that after Cornserve was announced, other omni-serving efforts appeared. The important distinction claimed in the lecture is that Cornserve emphasizes the planning problem. It does not simply implement one omni-model serving path; it tries to automatically decide the best deployment strategy.

Common Misconceptions

Misconception 1: Monolithic Serving is Always Bad

Monolithic serving can be best for some workloads. If components are small, communication overhead is high, or request shapes favor locality, monolithic deployment may outperform disaggregation.

Correct view:

Disaggregate only when the measured benefit exceeds the overhead.

Misconception 2: The Encoder is Always the Bottleneck

In many multimodal input models, encoders are smaller than the LM. For some workloads, the LM decode phase dominates. For omni models, an output generator such as an audio generator may be the actual bottleneck.

Correct view:

Bottlenecks are model-specific and workload-specific.

Misconception 3: Any-to-Any Serving is Just VLM Serving Plus One Component

Adding output generators changes the serving problem substantially. Output generators may have different batching behavior, latency profiles, and hardware utilization patterns. If they are slow, they can bottleneck monolithic serving.

Correct view:

Output modality generation introduces a new scheduling and scaling dimension.

Misconception 4: Continuous Batching Automatically Solves Everything

Continuous batching helps only within the component that can batch requests. If a monolithic system prevents the LM and generator from batching independently,

the system can still underperform.

Correct view:

Continuous batching must be combined with component-aware orchestration.

Misconception 5: Request Branching is Enough

Simple Python branching can decide which components are used, but it does not automatically expose a distributed invocation graph or independent parallel execution opportunities.

Correct view:

The runtime needs a record of component invocations and dependencies.

Misconception 6: Kernel Optimization is the Only Path to Faster Multimodal Serving

Kernel optimization matters, especially for generators and edge deployments. However, this lecture focuses on system-level scheduling, batching, disaggregation, and planning. Large gains can come from orchestration rather than from a new CUDA kernel.

Correct view:

Serving efficiency is a combination of kernels, batching, placement, routing, and resource allocation.

Practical Takeaways

1. **Model structure matters.** Treat any-to-any models as graphs of components, not as one opaque neural network.
2. **Request mixtures matter.** A workload with mostly text requests and rare audio outputs should be deployed differently from a workload with frequent audio generation.
3. **Component capacities matter.** Encoders, LMs, and generators can differ by large factors in throughput.
4. **Disaggregation is a tool, not a rule.** It improves some workloads and hurts others.
5. **Continuous batching should happen at the right level.** LM decode and audio generation may each need independent batching.

6. **Planning is essential.** Manual deployment choices do not scale across many model and workload combinations.
7. **Reuse existing serving engines.** Cornserve can rely on vLLM, SGLang, or TensorRT-LLM for LM execution while focusing on orchestration.
8. **Profiling beats intuition.** The lecture’s sweeps show that performance cannot be reliably predicted by simple rules.
9. **Output generators are emerging bottlenecks.** Audio and image generation components deserve serious systems attention.
10. **Hybrid deployments can win.** The best plan may route different fractions of traffic through different paths.

Project or Experiment Ideas

Experiment 1: Monolithic Versus Disaggregated Toy VLM

Implement a toy vision-language serving simulation with:

Image Encoder $\rightarrow$ LM.

Assign synthetic service times to the encoder and LM. Compare:

- monolithic serving;
- separate encoder and LM queues;
- hybrid routing.

Measure throughput and mean latency as image request probability varies.

Experiment 2: Prefill-Decode Disaggregation Simulator

Build a discrete-event simulator for LLM inference with prefill and decode phases. Model prefill as compute-heavy and decode as memory-heavy. Compare:

- unified worker;
- dedicated prefill worker plus dedicated decode worker;
- varying KV-transfer overhead.

Plot latency versus request arrival rate.

Experiment 3: Component Bottleneck Analysis

Create synthetic components with capacities:

$$\mu_{\text{image}} = 100, \quad \mu_{\text{LM}} = 40, \quad \mu_{\text{audio}} = 10.$$

Vary the fraction of audio-output requests from 0 to 1. Compute required replicas for stability:

$$x_i = \left\lceil \frac{\lambda_i}{\mu_i} \right\rceil.$$

Observe how the optimal allocation changes.

Experiment 4: Multicommodity Routing Solver

Implement a small integer linear program with request classes and paths. Use Python with a solver library and formulate:

$$\min \sum_e c_e x_e$$

subject to flow and capacity constraints. Use toy paths such as:

text $\rightarrow$ LM,

image plus text $\rightarrow$ Encoder $\rightarrow$ LM,

image plus text $\rightarrow$ Monolithic.

Compare solver output against hand-picked strategies.

Experiment 5: CUDA Microbenchmark for Decode-Like Memory Traffic

Write a CUDA kernel that streams through a large KV-cache-like tensor and computes dot products for one query per request. Vary sequence length S and batch size B . Measure effective bandwidth.

The approximate operation is:

```
__global__ void kv_dot_kernel(
    const float* __restrict__ q,
    const float* __restrict__ k,
    float* __restrict__ out,
    int batch,
    int seq_len,
    int dim
) {
    int b = blockIdx.x;
```



```

int s = blockIdx.y;
int tid = threadIdx.x;

extern __shared__ float partial[];

float acc = 0.0f;
for (int d = tid; d < dim; d += blockDim.x) {
    acc += q[b * dim + d] *
        k[(b * seq_len + s) * dim + d];
}

partial[tid] = acc;
__syncthreads();

for (int stride = blockDim.x / 2; stride > 0; stride /= 2) {
    if (tid < stride) {
        partial[tid] += partial[tid + stride];
    }
    __syncthreads();
}

if (tid == 0) {
    out[b * seq_len + s] = partial[0];
}
}

```

This kernel is intentionally simple. It maps one block to one batch element and one sequence position. Threads reduce over the head dimension. The experiment helps build intuition for why decode performance depends heavily on memory bandwidth and batching.

Final Mental Model

Cornserve is best understood as a **component-aware operating system for any-to-any model serving**. A modern multimodal model is not a single uniform computation. It is a graph of encoders, language-model stages, and output generators. Each component has different speed, memory usage, batching behavior, and GPU requirements. Each request activates only part of the graph.

A monolithic server treats the whole model as one object. Cornserve treats the model as a set of deployable tasks. It records which tasks each request needs, dispatches them to distributed executors, profiles component capacities, and uses a planner to decide how many replicas to allocate and how traffic should flow.

The essential performance equation is:

Good Serving = Right Components+Right Batching+Right Replication+Right Routing+Right Hardware Pla

For text-only LLMs, the main split is prefill versus decode. For multimodal input models, the split expands to encoder, prefill, and decode. For any-to-any models, the split expands again to include output generators. Cornserve’s key insight is that this expansion should be handled by a general task graph and planner rather than a new hand-built serving system for every model family.

The future of multimodal inference serving is not one universal monolithic engine. It is a planner-driven runtime that understands model components, request heterogeneity, and hardware bottlenecks well enough to choose the serving architecture automatically.